
IIC2523

Sistemas Distribuidos

— Hernán F. Valdivieso López —
(2026 - 2 / Clase 10)

Tolerancia a fallos

¿Qué es un fallo? ¿Cómo detectarlo? ¿Qué hacer?

Temas de la clase

1. Conceptos fundamentales y tipos de fracasos
2. *Failure Detection* y *Failure masking*
3. Teorema de Imposibilidad de Fischer, Lynch, Patterson

Conceptos fundamentales

Confiabilidad

Defecto, error y falla ¿son lo mismo?

Tipos de fallas

Conceptos fundamentales

- ◆ Un sistema distribuido tolerante a fallos es un sistema con una alta confiabilidad.

Conceptos fundamentales

- ◆ Un sistema distribuido tolerante a fallos es un sistema con una alta **confiabilidad**.

El grado en que un sistema informático puede ser utilizado de forma esperada.

Conceptos fundamentales

- ◆ Un sistema distribuido tolerante a fallos es un sistema con una alta **confiabilidad**.

El grado en que un sistema informático puede ser utilizado de forma esperada.

- ◆ Para lograr una alta confiabilidad. Hay varias propiedades que se desean cumplir:

- ✓  Disponibilidad

- ✓  Fiabilidad

- ✓  Seguridad

- ✓  Mantenibilidad

Conceptos fundamentales - Propiedades Confiabilidad

Propiedad de **Disponibilidad**



- ◆ Un sistema esté listo para ser usado inmediatamente, o la probabilidad de que esté operando correctamente en un momento dado.
- ◆ Por ejemplo, si un servidor tiene un 5% de probabilidad de fallar, dos servidores replicados aumentan la disponibilidad al 99.75%.

Propiedad de **Fiabilidad**



- ◆ Un sistema puede funcionar continuamente sin fallos durante un período de tiempo considerable.
- ◆ A diferencia de la disponibilidad, la fiabilidad mide el rango de tiempo que se puede ocupar sin fallos.

Conceptos fundamentales - Propiedades Confiabilidad

Propiedad de **Seguridad** 

- ◆ Si un sistema falla, no se producirán consecuencias catastróficas.

Propiedad de **Mantenibilidad** 

- ◆ La facilidad con la que un sistema fallido puede ser reparado después de una falla.

Conceptos fundamentales - Propiedades Confiabilidad

Ejemplos (Identificar el principal)

- ◆ Un sistema que administra una central nuclear exige una alta _____.
- ◆ Un código que ocupa varias librerías, pero está todo modularizado para poder cambiar una librería si es que falla. Eso garantiza una alta _____.
- ◆ Un sistema que funciona perfecto durante el día, pero de 3 a 6 de la madrugada entra en mantención y nadie puede acceder tiene una alta _____ y _____.
- ◆ Un aplicación para ver el clima debería tener una alta _____.
- ◆ Una plataforma para rendir un control de 5 minutos, donde tienes 1 semana para hacerla y 1 solo intento, debería tener una alta _____ y _____.

Conceptos fundamentales - Propiedades Confiabilidad

Ejemplos (Identificar el principal)

- ◆ Un sistema que administra una central nuclear exige una alta Seguridad.
- ◆ Un código que ocupa varias librerías, pero está todo modularizado para poder cambiar una librería si es que falla. Eso garantiza una alta Mantenibilidad.
- ◆ Un sistema que funciona perfecto durante el día, pero de 3 a 6 de la madrugada entra en mantención y nadie puede acceder tiene una alta Fiabilidad y Disponibilidad.
- ◆ Un aplicación para ver el clima debería tener una alta Disponibilidad.
- ◆ Una plataforma para rendir un control de 5 minutos, donde tienes 1 semana para hacerla y 1 solo intento, debería tener una alta Fiabilidad y Seguridad.

Conceptos fundamentales - Propiedades Confiabilidad

Ejemplos (Identificar el principal)

- ◆ Un sistema que administra una central nuclear exige una alta Seguridad.
- ◆ Un código que ocupa varias librerías, pero está todo modularizado para poder cambiar una librería si es que falla. Eso garantiza una alta Mantenibilidad.
- ◆ Un sistema que funciona perfecto durante el día, pero de 3 a 6 de la madrugada entra en mantención y nadie puede acceder tiene una alta Fiabilidad y Disponibilidad.
- ◆ Un aplicación para ver el clima debería tener una alta Disponibilidad.
- ◆ Una plataforma para rendir un control de 5 minutos, donde tienes 1 semana para hacerla y 1 solo intento, debería tener una alta Fiabilidad y Seguridad.

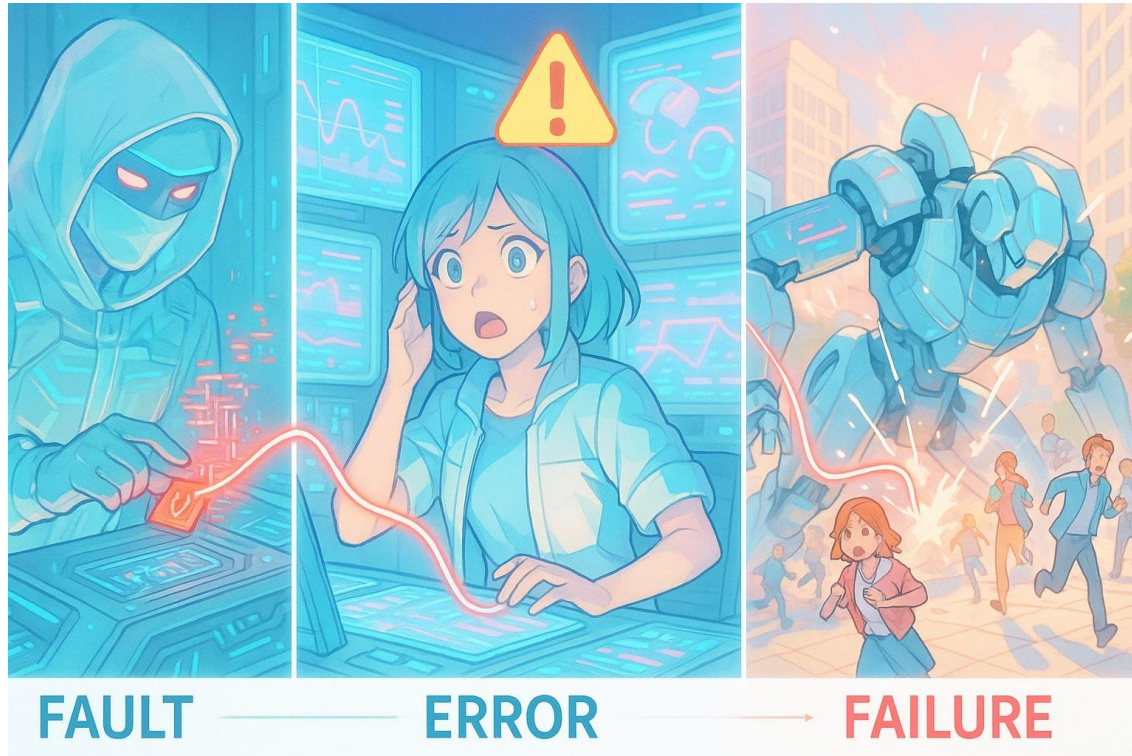
! Se espera que un sistema cumpla la mayor cantidad de estas propiedades !

Conceptos fundamentales - Defecto, Error y Falla

¿Son 3 conceptos para describir lo mismo?

Conceptos fundamentales - Defecto, Error y Falla

¿Son 3 conceptos para describir lo mismo? No.

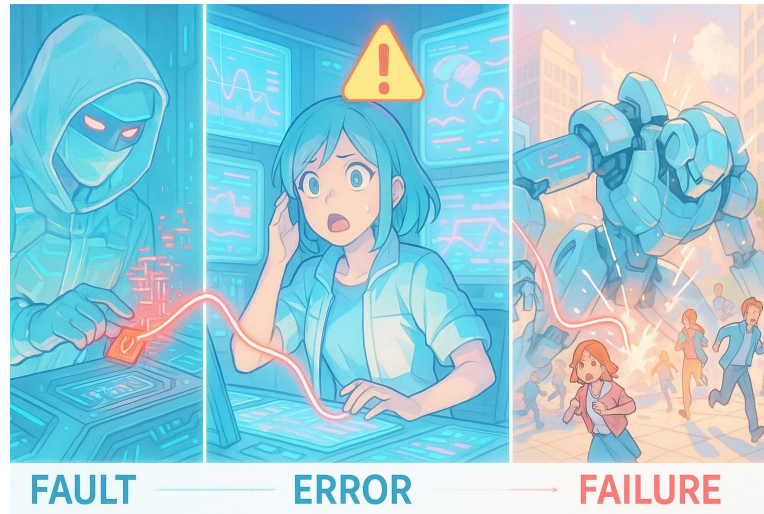


Conceptos fundamentales - Defecto, Error y Falla

¿Son 3 conceptos para describir lo mismo? No.

◆ Cada concepto describe un aspecto distinto de una operación que no logró ser ejecutada con éxito en el sistema. Se define la siguiente cadena de causalidad:

◆ **Defecto/Culpa (*Fault*) → Error → Falla (*Failure*)**



Conceptos fundamentales - Defecto, Error y Falla

¿Son 3 conceptos para describir lo mismo? No.

- ◆ Cada concepto describe un aspecto distinto de una operación que no logró ser ejecutada con éxito en el sistema. Se define la siguiente cadena de causalidad:
 - ◆ **Defecto/Culpa (*Fault*) → Error → Falla (*Failure*)**
- ◆ En términos simples, el defecto/culpa es la causa de un error. El error es justamente lo que ocurre dentro del sistema y la falla es el estado final de la operación.
 - ◆ Por ejemplo, haces una calculadora en Python que divide dos números y le entrega un 5 de numerador y luego un 0 de denominador.
 - ◆ Fault: entrega un 0 como denominador y no tener validación ante eso.
 - ◆ Error: `ZeroDivisionError`
 - ◆ Failure: se cierra el código, no divide, etc.

Conceptos fundamentales - Defecto, Error y Falla

¿Son 3 conceptos para describir lo mismo? No.

- ◆ Cada concepto describe un aspecto distinto de una operación que no logró ser ejecutada con éxito en el sistema. Se define la siguiente cadena de causalidad:
 - ◆ **Defecto/Culpa (*Fault*) → Error → Falla (*Failure*)**
- ◆ En términos simples, el defecto/culpa es la causa de un error. El error es justamente lo que ocurre dentro del sistema y la falla es el estado final de la operación.
- ◆ **Un error no necesariamente lleva a la falla** si el sistema posee mecanismos para tolerar la falla.
 - ◆ Por ejemplo, servidores de respaldo, algoritmos que logran finalizar incluso si se cae el nodo, *try/except*, etc.

Conceptos fundamentales - Defecto, Error y Falla

¿Son 3 conceptos para describir lo mismo? No.

- ◆ Cada concepto describe un aspecto distinto de una operación que no logró ser ejecutada con éxito en el sistema. Se define la siguiente cadena de causalidad:

- ◆ El objetivo principal es la **tolerancia a fallos**

- ◆ En todo momento, un sistema puede proporcionar sus servicios incluso en presencia de errores o fallas.

- ◆ Un sistema distribuido debe estar diseñado para **absorber el golpe**.

tolerar la falla.

- ◆ Por ejemplo, servidores de respaldo, algoritmos que logran finalizar incluso si se cae el nodo, *try/except*, etc.

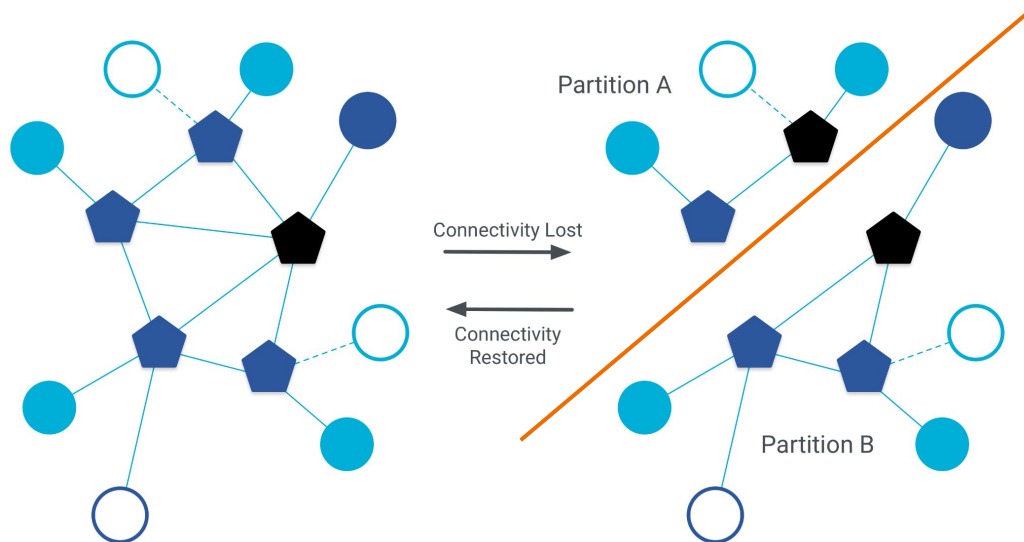
Conceptos fundamentales - Defecto, Error y Falla

- ◆ En un sistema distribuido, pueden existir distintos tipos de fallas.
 - ◆ Partición de red
 - ◆ Caída de uno o más nodos
 - ◆ Omisión de mensaje
 - ◆ Tiempo de respuesta
 - ◆ Respuesta incorrecta

Conceptos fundamentales - Defecto, Error y Falla

◆ Particiones de Red (*network partitions*)

- ◆ La red puede dividirse en subgrupos donde la comunicación entre ellos es imposible, incluso si los nodos individuales dentro de los subgrupos siguen funcionando.
- ◆ Estrategia tolerante: utilizar un mecanismo de consenso que requiera una mayoría de nodos para continuar operando.



Conceptos fundamentales - Defecto, Error y Falla



Caída de uno o más nodos (*crash failure*)

- ♦ El nodo funcionaba correctamente hasta que en algún momento se apaga o se cuelga y deja de ejecutar su programa.
- ♦ Estrategia tolerante: disponer de más nodos que puedan suplir el rol del nodo caído.

Conceptos fundamentales - Defecto, Error y Falla

- ◆ Caída de uno o más nodos (*crash failure*)
 - ◆ El nodo funcionaba correctamente hasta que en algún momento se apaga o se cuelga y deja de ejecutar su programa.
 - ◆ Estrategia tolerante: disponer de más nodos que puedan suplir el rol del nodo caído.
- ◆ Omisión de mensaje (*omission failure*)
 - ◆ Un mensaje que debería ser enviado o recibido no llega a su destino.
 - ◆ Estrategia tolerante: re-transmitir mensajes y usar confirmación (ACKs).

Conceptos fundamentales - Defecto, Error y Falla

◆ Tiempo de respuesta (*timing failure*)

- ◆ El servidor responde, pero lo hace fuera del intervalo de tiempo esperado.
- ◆ Estrategia tolerante: utilizar *timeouts* para detectar respuestas que llegan demasiado tarde.

Conceptos fundamentales - Defecto, Error y Falla

◆ Tiempo de respuesta (*timing failure*)

- ◆ El servidor responde, pero lo hace fuera del intervalo de tiempo esperado.
- ◆ Estrategia tolerante: utilizar *timeouts* para detectar respuestas que llegan demasiado tarde.

◆ Respuesta incorrecta (*response failure*)

- ◆ El servidor responde, pero de forma incorrecta a lo esperado. Puede ser:
 - ◆ *Value failure*: El valor de la respuesta está mal.
 - ◆ *State-transition failure*: El nodo sigue un flujo de control incorrecto, es decir, actúa fuera del orden esperado.
- ◆ Estrategia tolerante: utilizar replicación y comparar las respuestas de varios nodos para detectar resultados incorrectos.

Conceptos fundamentales - Defecto, Error y Falla

- ◆ En un sistema distribuido, pueden existir distintos tipos de fallas.
 - ◆ Partición de red
 - ◆ Caída de uno o más nodos
 - ◆ Omisión de mensaje
 - ◆ Tiempo de respuesta
 - ◆ Respuesta incorrecta

El objetivo final es diseñar estrategias que permitan al sistema tolerar estas fallas y continuar funcionando correctamente.

Failure Detection y Failure masking

Mecanismos para detectar un
fracaso

Mecanismos para ser tolerante
a fallos

Failure Detection

- ◆ Existen diferentes mecanismos para detectar el fallo de un sistema. Aunque no pueden precisar el tipo de fallo.
 - ◆ **Sondas (*probes*):** mecanismo activo donde se envían mensajes del tipo "¿sigues vivo?" a los diferentes nodos.
 - ◆ **Latidos (*heartbeats*):** mecanismo pasivo donde se espera mensajes de otro nodo para entender que "sigue vivo". Funciona solo si se garantiza un buen canal de comunicación.
 - ◆ **Tiempo de espera (*timeouts*):** las mensajes poseen un tiempo máximo de respuesta, si no se cumple se sospecha que el nodo ha fallado. Incluso si un mensaje no espera una respuesta, se puede obligar al nodo a responder con un "ACK" (confirmar que recibió el mensaje).
 - También conocido como *request-response*: cada mensaje enviado exige confirmación en un tiempo determinado.

Failure Detection

Desafíos

- ◆ En sistemas donde no hay límites en las velocidades de ejecución o los retrasos de mensajes, un *timeout* solo indica que un proceso no responde, pero...
 - ◆ ¿Se bloqueó? ¿está lento? ¿el mensaje se perdió?
 - ◆ Esto puede generar falsos positivos (creer que el nodo falló cuando fue la red).

Failure Detection

Desafíos

- ◆ En sistemas donde no hay límites en las velocidades de ejecución o los retrasos de mensajes, un *timeout* solo indica que un proceso no responde, pero...
 - ◆ ¿Se bloqueó? ¿está lento? ¿el mensaje se perdió?
 - ◆ Esto puede generar falsos positivos (creer que el nodo falló cuando fue la red).
- ◆ Por lo mismo, existen los mecanismos "eventualmente perfectos" que intentan adaptarse al sistema y sus posibles fallas.
 - ◆ Ajustar *timeout* para los distintos nodos o enviar más de una petición para asegurar que al menos 1 mensaje llegue o para contrastar respuestas.

Failure masking

- ◆ Un sistema tolerante a fallos es aquel que puede continuar el rendimiento correcto de sus tareas especificadas en presencia de fallos de hardware y/o software.
- ◆ Su objetivo principal es evitar que los fallos generen un fracaso del sistema.
- ◆ La **redundancia** es un concepto clave para lograr este objetivo.

Failure masking

- ◆ Un sistema tolerante a fallos es aquel que puede continuar el rendimiento correcto de sus tareas especificadas en presencia de fallos de hardware y/o software.
- ◆ Su objetivo principal es evitar que los fallos generen un fracaso del sistema.
- ◆ La **redundancia** es un concepto clave para lograr este objetivo.
 - ◆ Implica la **adición** de información, recursos o tiempo **más allá de lo necesario** para el funcionamiento normal del sistema.
 - ◆ Sin embargo, la redundancia puede impactar el rendimiento, tamaño, peso y consumo de energía de un sistema.

Failure masking

- ◆ Un sistema tolerante a fallos es aquel que puede continuar el rendimiento correcto de sus tareas especificadas en presencia de fallos de hardware y/o software.
- ◆ Su objetivo principal es evitar que los fallos generen un fracaso del sistema.
- ◆ La **redundancia** es un concepto clave para lograr este objetivo.
 - ◆ Implica la **adición** de información, recursos o tiempo **más allá de lo necesario** para el funcionamiento normal del sistema.
 - ◆ Sin embargo, la redundancia puede impactar el rendimiento, tamaño, peso y consumo de energía de un sistema.
 - ◆ Se distinguen 4 tipos de redundancia: *Hardware*, *Información*, *Tiempo*, *Software*.

Failure masking - Redundancia

Redundancia de *hardware* (enfoque **pasivo**)

- ◆ Se ignora la ocurrencia de una falla.
- ◆ **TMR - Triple Modular Redundancy**: se triplica el *hardware* y luego se hace votación para determinar la salida del sistema.
 - ◆ No se preocupa por determinar la falla y corregir, solo deja que el Quórum escoja, idealmente, la respuesta correcta.
 - ◆ Se puede extender a N *hardware* (con N impar) para detectar más fallas simultáneas.

Failure masking - Redundancia

Redundancia de *hardware* (enfoque **activo**)

- ◆ Detectan el componente fallado y lo reparan o reemplazan.
- ◆ **Standby Sparing**: Uno o más módulos están operativos y hay otros de respaldo (*spares*). Si se detecta y localiza un fallo, el módulo defectuoso se retira y se reemplaza por un repuesto. El defectuoso luego se intenta reparar.
 - ◆ **Hot Standby Sparing**: Los repuestos operan en sincronía con los módulos en línea, listos para tomar el control de inmediato.
 - ◆ **Cold Standby Sparing**: Los repuestos están apagados hasta que se necesitan.

Failure masking - Redundancia

Redundancia de información

- ◆ Se añade información adicional a los datos para detectar y/o reconstruir el dato.
- ◆ Algunos ejemplos son:
 - ◆ **Código duplicado**: toda la información está duplicada N veces.
 - ◆ **Checksum**: bloque con la suma de todos los *bytes*. Existen diferentes fórmulas para sumar.
 - ◆ **Código Hamming**: se agregan *bits* extras de paridad para detectar si un *bit* de la información original falló y corregirlo.

Failure masking - Redundancia

Redundancia de tiempo

- ◆ Reducir el *hardware* extra a expensas de usar tiempo adicional.
- ◆ Se realiza la misma operación dos o más veces y se comparan los resultados. Si hay discrepancia, se repite la computación para ver si el error desaparece.
- ◆ Se incluye concepto de votación entre ejecuciones para determinar el resultado a enviar.
- ◆ Para detectar fallos permanentes, se realizan operaciones con resultados esperados.

Failure masking - Redundancia

Redundancia de *software*

- ◆ Un nodo posee más programas/procesos para asegurar el correcto funcionamiento.
- ◆ Incluir programas para verificar información con conocimiento previo o verifican el estado del sistema (pruebas de memoria, prueba a la ALU, etc).
- ◆ También se pueden recurrir a técnicas a la redundancia de *hardware*, pero ahora con programas/procesos.
 - Varios procesos concurrentes y luego se vota por la respuesta.
 - Disponer de procesos de respaldo.

Teorema de Imposibilidad de Fischer, Lynch, Patterson

Teorema de Imposibilidad de Fischer, Lynch, Patterson

- ◆ Teorema creado en 1985 por Michael J. Fischer, Nancy A. Lynch y Michael S. Paterson.
- ◆ Resultado fundamental en la teoría de los sistemas distribuidos.
- ◆ Ganó el premio *Dijkstra* el 2001.
 - ◆ El premio se otorga a trabajos destacados sobre los principios de la computación distribuida, cuya importancia e impacto en la teoría y/o práctica de la computación distribuida haya sido evidente durante al menos una década.
 - ◆ El premio incluye una dotación de \$2000 dólares 

Teorema de Imposibilidad de Fischer, Lynch, Patterson

- ◆ **Sistema asíncrono:** No existe ninguna cota superior conocida (ni demostrable) sobre cuánto puede tardar un mensaje en llegar, ni sobre cuánto puede tardar un proceso en ejecutar un paso.

Teorema de Imposibilidad de Fischer, Lynch, Patterson

- ◆ **Sistema asíncrono:** No existe ninguna cota superior conocida (ni demostrable) sobre cuánto puede tardar un mensaje en llegar, ni sobre cuánto puede tardar un proceso en ejecutar un paso.

Teorema

*En un sistema completamente asíncrono, es **imposible garantizar** el consenso (todos los nodos queden de acuerdo) si incluso un solo nodo es propenso a caídas.*

Teorema de Imposibilidad de Fischer, Lynch, Patterson

- ◆ **Sistema asíncrono:** No existe ninguna cota superior conocida (ni demostrable) sobre cuánto puede tardar un mensaje en llegar, ni sobre cuánto puede tardar un proceso en ejecutar un paso.

Teorema

*En un sistema completamente asíncrono, es **imposible garantizar** el consenso (todos los nodos queden de acuerdo) si incluso un solo nodo es propenso a caídas.*

- ◆ **Justificación:** en un sistema asíncrono, no se puede distinguir entre un nodo muy lento y un nodo que ha fallado.

Teorema de Imposibilidad de Fischer, Lynch, Patterson

Teorema

*En un sistema completamente asíncrono, es **imposible garantizar** el consenso (todos los nodos queden de acuerdo) si incluso un solo nodo es propenso a caídas.*

- ◆ FLP establece un límite fundamental: en un sistema completamente asíncrono, no podemos garantizar **simultáneamente** consenso total y tolerancia a fallas por caídas.
- ◆ Debemos ser rigurosos con los supuestos que hacemos al diseñar algoritmos distribuidos.

Teorema de Imposibilidad de Fischer, Lynch, Patterson

Estrategias para la Tolerancia a Fallas considerando teorema:

◆ Asumir Sincronía Parcial

- ◆ Incorporar límites de tiempo al sistema asíncrono.
- ◆ Por ejemplo, utilizar *timeouts* y detectar que un nodo probablemente ha fallado.

◆ Relajar las garantías

- ◆ Exigir un *quórum* para tomar decisiones, aceptando que el sistema pueda dejar de avanzar cuando no exista una mayoría disponible.
- ◆ Aceptar que algunas propiedades (*termination*, *agreement*) no puedan garantizarse en todas las situaciones.

Poniendo a prueba lo que hemos aprendido 🧐

¿Cuáles de las siguientes afirmaciones sobre tolerancia a fallos en sistemas distribuidos es **correcta**?

- a. Un sistema con alta disponibilidad necesariamente tiene alta fiabilidad.
- b. Un sistema tolerante a fallos se asegura que no ocurra ningún fallo.
- c. Se pueden reparar mensajes dañados aplicando correctamente redundancia de información.
- d. Una falla de tiempo se refiere a que el nodo no responderá aunque se le de todo el tiempo posible.
- e. Es requisito primordial que un sistema tolerante a fallo pueda detectar los fallos que ocurren.

Poniendo a prueba lo que hemos aprendido 🙄

¿Cuáles de las siguientes afirmaciones sobre tolerancia a fallos en sistemas distribuidos es **correcta**?

- a. Un sistema con alta disponibilidad necesariamente tiene alta fiabilidad.
- b. Un sistema tolerante a fallos se asegura que no ocurra ningún fallo.
- c. Se pueden reparar mensajes dañados aplicando correctamente redundancia de información.**
- d. Una falla de tiempo se refiere a que el nodo no responderá aunque se le de todo el tiempo posible.
- e. Es requisito primordial que un sistema tolerante a fallo pueda detectar los fallos que ocurren.

Próximos eventos

Próxima clase (Vuelta de receso)

- ◆ Si distintos usuarios pueden observar estados diferentes de un mismo sistema distribuido
 - ¿Qué reglas debe seguir el sistema para definir qué valores son "correctos" en cada momento?
 - Estudiaremos el concepto de "Modelo de consistencia"

Evaluación

- ◆ **Control 10 - Obligatorio** - 2 preguntas sobre la clase de hoy. Se puede responder hasta este viernes a las 20:00.

IIC2523

Sistemas Distribuidos

— Hernán F. Valdivieso López —
(2026 - 2 / Clase 10)
